GUÍA DE PROGRAMACIÓN

Para qué sirve `Console.Write` , `Console.WriteLine` y `Console.ReadLine`

`Console.Write` y `Console.WriteLine` sirven para mostrar cualquier texto en la pantalla. La diferencia principal es que `Console.Write` deja el cursor en la misma linea, y `Console.WriteLine` te lleva a la siguiente linea de forma automática en cuanto terminas de escribir. `Console.ReadLine` sirve para que el usuario pueda escribir algo en el programa desde su teclado.

CÓDIGO

```
Console.Write("Hola");
Console.Write(", ");
Console.Write("Diana!");

Console.WriteLine("Hola, Diana.");
Console.WriteLine("Bienvenida#");

Console.Write("Escribe tu nombre: ");
string nombre = Console.ReadLine();
Console.WriteLine("Hola " + nombre);
```

SECUENCIAS DE ESCAPE

Una secuencia de escape se usa para representar caracteres especiales, saltos de linea (Se usan dentro de las cadenas de texto string)

```
\n - Crea un salto de linea.
\t - Inserta una tabulación horizontal (Espacio grande).
\" o \' - Permite usar comillas dobles o simples dentro de un texto delimitado dentro de ellas sin
necesidad de romper el código.
\\ - Muestra una barra invertida en la pantalla.
```

COLORES DE FONDO Y TEXTO

Los colores de fondo y texto sirven para dar alertas visuales instantáneas (como rojo para error o verde para éxito) y guiar la vista del usuario para que el programa sea más fácil de leer.

```
- Console.ForegroundColor - Sirve para cambiar el color del texto. Todo lo escrito en la linea
siguiente de este comando, tendrá el color que elija.
- Console.BackgroundColor - Sirve para cambiar el color del fondo de las letras. Funciona como un
marcador y resaltador de textos.
```

```
- Console.Color.(Color) - Es la lista de colores predefinidos. Por ejemplo: "Console.Color.Yellow", "Console.Co
"Dark blue", "Green", etc.
- Console.ResetColor() - Apaga el color que estabas usando y regrsa al color base. Te permite pintar
una palabra de un color y asegurarte de que la siguiente palabra vuelva a la normalidad
automáticamente.
```

TIPOS DE DATOS, VARIABLES Y CONSTANTES

Tipos de Datos explícitos e implícitos

```
TIPOS DE DATOS EXPLICITOS (Se especifica el tipo de dato antes del nombre).
    Ejemplos:
int age = 20;
string name = "John";
char gender = 'M';
bool isStudent = true;
float height = 1.75f;
double cash = 1000;
Dog rover = new Dog("Rover");

TIPOS DE DATOS IMPLÍCITOS (Se pone "var" y asume de forma automática el tipo de dato).
var age = 20;
var name = "John";
var gender = 'M';
var isStudent = true;
var height = 1.75f;
var cash = 1000;
var rover = new Dog("Rover");
```

VARIABLES Y CONSTANTES.

VARIABLES solo pueden contener un tipo de dato y su valor puede cambiar.

CONSTANTES su valor no cambia una vez asignado, se defince al inicio y van en mayusculas.

Ejemplos y uso:

```
VARIABLES
int age = 20;
string name = "John";
char gender = 'M';
bool isStudent = true;
float height = 1.75f;
double cash = 1000;

CONSTANTES
const int AGE = 20;
const string NAME = "John";
const char GENDER = 'M';
const bool IS_STUDENT = true;
const float HEIGHT = 1.75f;
const double CASH = 1000;
```

TIPOS DE DATOS NUMÉRICOS ENTEROS

ENTEROS: Se usan para recursos, vida de los personajes, edades puntajes, etc.

```
sbyte _ -128 a 127
byte _ 0 a 255
short _ -32,768 a 32,767
ushort _ 0 a 65,535
int _ -2,147,483,648 a 2,147,483,647
uint _ 0 a 4,294,967,295
long _ -9,223,372,036,854,775,808 a 9,223,372,036,854,775,807
ulong _ 0 a 18,446,744,073,709,551,615
```

TIPOS DE DATOS NUMÉRICOS DECIMALES

DECIMALES/FLOTANTES: Se usan para coordenadas. distancias, medidas, bonus y hit chance.

```
float _ ±1.5x10−45 a ±3.4x1038
double _ ±5.0×10−324 a ±1.7×10308
decimal _ ±1.0x10-28 a ±7.9228x1028
```

TRYPARSE

Convierte un texto (string) a número entero (int) sin romper el programa si se escriben letras.

```
Ejemplo:

Console.Write("Escribe tu edad: ");
string entrada = Console.ReadLine();

// TryParse intenta convertir 'entrada' y guarda el número en 'edad'
int edad;
bool funciono = int.TryParse(entrada, out edad);
```

BOOLEANOS

Se usa en condicionales

```
True y false
```

CONDICIONALES

```
if (si se cumple)
else (si no se cumple nada)
else if (si no se cumple lo anterior, evalua lo mas reciente). Ej: No tenia un boleto para entrar, pero si dine
switch (coincide exactamente con el valor de la variable)
```

CICLOS: Sirven para repetir un bloque de código

```
while (Sigue siempre y cuando se cumpla la condición)
do-while (Ejecuta siempre la primera vez, y luego pregunta al hacer algo)
for (Se usa cuando se sabe cuantas veces se repetirá algo)
foreach (Va de inicio a fin)
```

OPERACIONES

Las operaciones aritméticas siempre se usan para los tipos de datos numéricos.

`División entera:` Solo aplica para tipos de datos enteros.

`Divisiön real:` Se aplica cuando algun numero es decimal y su resultado siempre es decimal.

```
JERARQUÍA DE OPERACIONES
Agrupación _ ()
Unary Negative _ -
Multiplicación y División _ *, /
Resto y Módulo _ %
Suma y Resta _ +, -

Adición Unaria (Prefijo) _ ++
Resta Unaria (Prefijo) _ --
Adición Unaria (Sufijo) _ ++
Resta Unaria (Sufijo) _ --
```

Ejemplos:

```
Suma _ Ejemplo: num1 + num2:  5 + 5 = 10
Resta _ Ejemplo: num1 - num2: 7 - 5 = 2
Multiplicación _ Ejemplo: num1 * num2: 5F * 5 = 25F
División entera _ Ejemplo: num1 / num2: 5 / 2 = 2
División real _ Ejemplo: num1 / num2: 10 / 2F = 5F
Resto o módulo _ Ejemplo: num1 % num2: 2 % 3 = 2

Adición Unaria (Prefijo) ++ _ Se suma 1 antes de cualquier acción _ Ejemplo: ++num ++9 10
Resta Unaria (Prefijo) -- _ Se resta 1 antes de cualquier acción _ Ejemplo: --num --9 8
Adición Unaria (Sufijo) ++ _ Se suma 1 después de cualquier acción _ Ejemplo: num++ 9++ 10
Resta Unaria (Sufijo) -- _ Se resta 1 después de cualquier acción _ Ejemplo: num-- 9-- 8
Unary Plus + _ Es equivalente a multiplicar por +1 _ Ejemplo: +num +(-9) -9
Unary Negative - _ Es equivalente a multiplicar por -1 _ Ejemplo: -num -(-9) 9
```

OPERACIONES RELACIONALES Y LÓGICAS

Las operaciones relacionales y lógicas nos permiten comparar operciones y poder obtener un resultado, ya sea verdadero o falso, y este resultado siempre será un bool.

```
JERARQUÍA DE OPERACIONES:
Relacionales <, >, >=, <=
Igualdad ==,!=
NOT !
AND &&
OR ||

OPERACIONES RELACIONALES:
Igualdad == Determina si dos valores son iguales. num1 == num2 4 == 2 => false
Diferente != Determina si dos valores son diferentes. num1 != num2 4 != 2 => true
Mayor que > Determina si un valor es mayor que otro. num1 > num2 4 > 2 => true
Menor que < Determina si un valor es menor que otro num1 < num2 4 < 2 => true
Mayor o igual que >= Determina si un valor es mayor o igual que otro. num1 >= num2 4 >= 2 => true
Menor o igual que <= Determina si un valor es menor o igual que otro. num1 <= num2 4 <= 2 => true"

OPERACIONES LÓGICAS:
AND && -true-, si ambos valores son -true-
OR || -true-, si cualquier valor es -true-
NOT ! Convierte el valor es su opuesto -true- <-> -false-
```

ANLISIS Y RESOLUCIÓN DE PROBLEMAS

1. Leer el problema e identificar los datos: Entradas y salidas
2. Escribir los pasos en psudocódigo antes de empezar con el algoritmo
3. Traducir el pseudocódigo a C#

DEBUGGING

¿Qué hacer si mi programa no funciona?

```
Lineas rojas (Errores) = El código está mal escrito y son errores críticos, puede ser algo como que falte un parentesis, comillas, ortografía, etc.
Lineas amarillas (Advertencias) = Pueden causar problemas o tal vez no, pero no evitan que el programa corra.
Lineas verdes (Sugerencias) = Sugerencias o recomendaciones de escritura o forma de aplciar los estandares.
```